



REAL-TIME ARCHITECTURE DOCUMENT

GameVerse — Esports Tournament Operations & Live Broadcast Platform

Version: 1.0 | June 2025 | ★ Critical Infrastructure Document

TABLE OF CONTENTS

1. Document Overview & Real-Time Philosophy
 2. Real-Time Architecture Overview
 3. STOMP WebSocket Infrastructure
 4. Complete Event Catalog
 5. Event Payload Specifications
 6. The Complete Real-Time Pipeline (Score → OBS)
 7. Connection Lifecycle & Authentication
 8. Reconnection Behavior & Recovery
 9. Data Synchronization Strategy
 10. Fallback Behavior
 11. Channel & Subscription Architecture
 12. OBS Overlay Real-Time System
 13. Security in Real-Time
 14. Performance & Scalability
 15. Error Handling in Real-Time
 16. Testing Real-Time Systems
-
-

SECTION 1 — DOCUMENT OVERVIEW & REAL-TIME PHILOSOPHY

1.1 Purpose

This document defines the **complete real-time architecture** for GameVerse — every WebSocket event, every payload schema, every reconnection behavior, every fallback strategy, and every data synchronization mechanism that powers live esports tournament operations.

Real-time is **not a feature** in GameVerse. It is the **foundation**. A score update that takes 30 seconds to appear on the leaderboard is a broken product in live esports. A credential that fails to deliver to one player causes a no-show. An overlay that freezes during the grand finale damages the broadcast.

1.2 Real-Time Requirements

HARD LATENCY REQUIREMENTS:

Event	Max Latency	Priority
Score → Leaderboard visible	< 3 seconds	● CRITICAL
Credential → Player screen	< 2 seconds	● CRITICAL
Match status change → all	< 2 seconds	● CRITICAL
Score → OBS Overlay update	< 3 seconds	● CRITICAL
Technical pause → teams	< 2 seconds	● CRITICAL
Dispute alert → director	< 3 seconds	● HIGH
Check-in update → command ctr	< 5 seconds	● HIGH
Stream health → broadcast dash	< 10 seconds	● MEDIUM
Analytics update	< 60 seconds	● LOW

1.3 Real-Time Consumer Map

WHO RECEIVES WHAT IN REAL-TIME:

EVENT SOURCE	CONSUMERS
Score Submitted	Director (verification alert)
Score Published	All players, viewers, OBS overlays
Leaderboard Updated	Everyone connected to tournament
Credential Released	ONLY 64 players in that match
Match Status Changed	All tournament subscribers
Technical Pause	All players + staff
Dispute Created	Director only
Check-in Updated	Director + referees only
Announcement Sent	All registered participants
Stream Health	Broadcast producer + director
Tournament Completed	All subscribers (prize notification)

1.4 Protocol Decision: STOMP over WebSocket

WHY STOMP:

Raw WebSocket gives you:

- └ A pipe. You manage everything: routing, subscriptions, auth.

STOMP gives you:

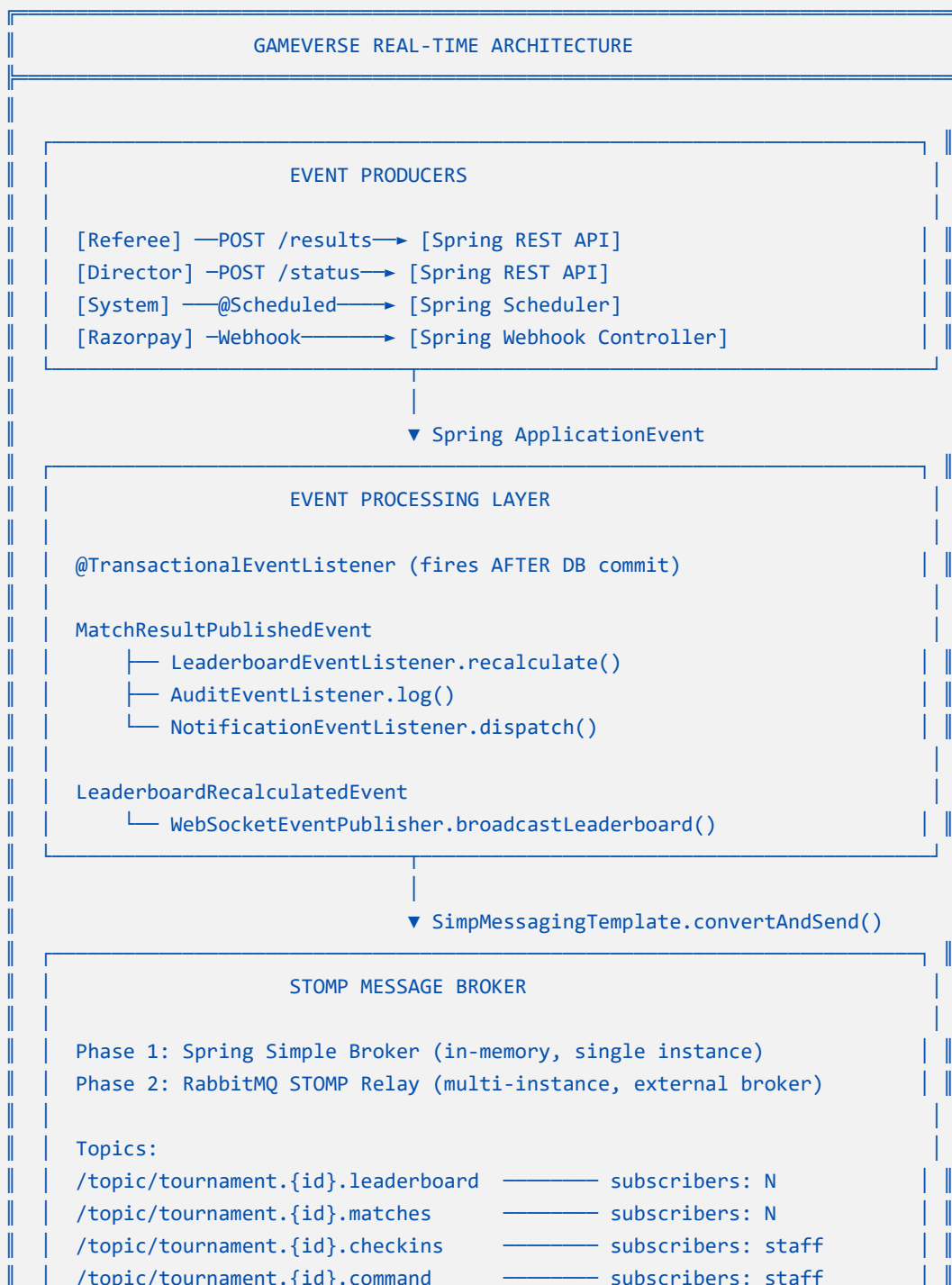
- └ Topic-based publish/subscribe (built-in)
- └ Destination-based message routing
- └ Message headers (auth, content-type, message-id)
- └ Receipt acknowledgment
- └ Built-in heartbeat protocol
- └ SockJS fallback for restricted networks (OBS, corporate)

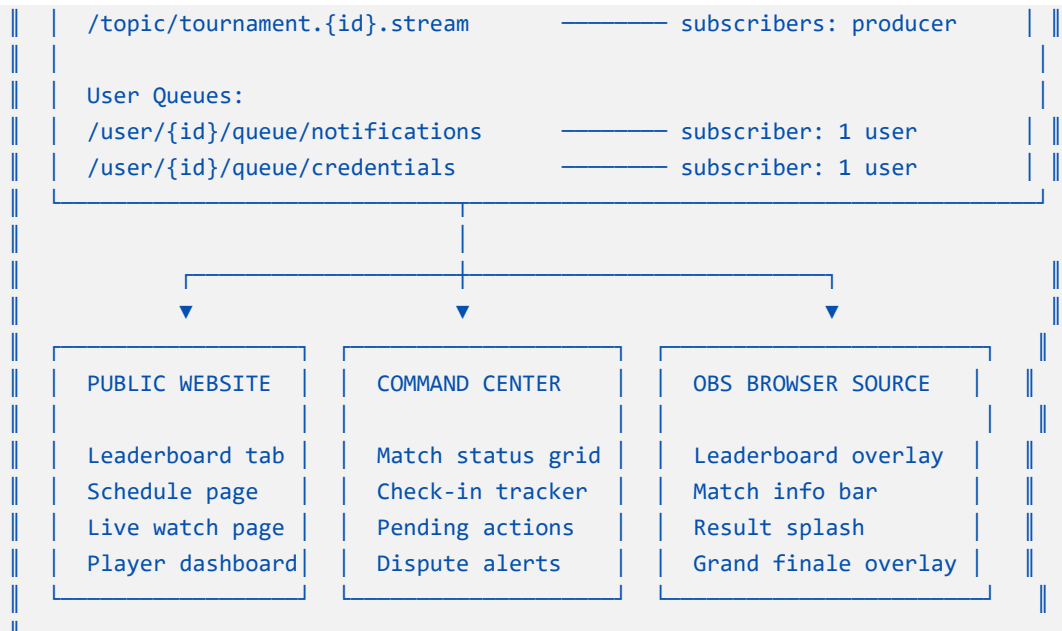
Spring Boot's native STOMP support means:

- └─ @MessageMapping → receives client → server messages
- └─ @SendTo("/topic/...") → broadcasts to all subscribers
- └─ SimpMessagingTemplate → server-initiated broadcasts
- └─ Spring Security integrates directly into STOMP handshake
- └─ No external dependency for MVP (Phase 1: in-memory broker)

SECTION 2 — REAL-TIME ARCHITECTURE OVERVIEW

2.1 Complete Real-Time System Map





2.2 The Master Pipeline: Score Update → OBS Overlay

COMPLETE JOURNEY OF A SCORE UPDATE (with timestamps):

- T+0ms Referee clicks "Submit Results" in Command Center
- T+10ms POST /api/v1/tournaments/{id}/matches/{id}/results
 → Spring Security validates JWT + RBAC
 → Bean Validation checks all placements unique, kills ≥ 0
- T+50ms @Transactional begins:
 → MatchResult entity saved to MySQL
 → 16 TeamMatchResult entities saved (one per team)
 → ScoringEngine.calculate() runs in-transaction:
 effective_kills = MIN(raw_kills, kill_cap)
 kill_points = effective_kills × kill_pts_each
 placement_points = lookup(placement, template)
 total_points = kill_points + placement_points
 → Match status → RESULT_SUBMITTED
- T+80ms @Transactional commits to MySQL
- T+85ms @TransactionalEventListener fires:
 (AFTER_COMMIT – guaranteed DB is consistent)
 MatchResultPublishedEvent consumed by:
 └─ LeaderboardEventListener (priority: HIGHEST)
 └─ AuditEventListener (priority: HIGH)
 └─ NotificationEventListener (priority: NORMAL)
- T+120ms LeaderboardEngine.recalculate(tournamentId):
 → SELECT SUM(total_points), COUNT(chicken_dinners),
 SUM(raw_kills) FROM team_match_results
 GROUP BY registration_id ORDER BY tiebreaker
 → LeaderboardEntry rows updated in MySQL
 → Previous ranks stored (for rank_change calculation)

→ Caffeine cache invalidated and refreshed

T+180ms LeaderboardRecalculatedEvent fires:
WebSocketEventPublisher.broadcastLeaderboard()
→ SimpMessagingTemplate.convertAndSend(
 "/topic/tournament.{id}.leaderboard",
 leaderboardPayload
)

T+185ms Spring STOMP broker routes message:
→ All subscribers on /topic/tournament.{id}.leaderboard
→ Message delivered via WebSocket frames

T+200ms PUBLIC WEBSITE (React):
→ useStompSubscription hook receives message
→ Zustand store updated with new leaderboard
→ React re-renders leaderboard table
→ CSS animations play: rows slide to new positions

T+210ms COMMAND CENTER (React):
→ Same STOMP subscription
→ CC leaderboard section updates
→ Pending actions badge refreshes

T+210ms OBS BROWSER SOURCE (Vanilla JS):
→ STOMP subscription receives same message
→ DOM updated: team names, points, rank numbers
→ CSS transition: rows animate to new positions
→ Rank arrows animate ↑ or ↓

TOTAL: ~210ms from submission to visible update

TARGET: < 3000ms ✅ (10x headroom at MVP scale)

SECTION 3 — STOMP WEBSOCKET INFRASTRUCTURE

3.1 Spring WebSocket Configuration

STOMP ENDPOINT CONFIGURATION:

WebSocket Endpoint: /ws
├─ Protocol: STOMP over WebSocket
├─ SockJS Fallback: /ws (with SockJS negotiation)
│ ├─ Used by OBS Browser Sources on restricted networks
│ └─ Fallback transports: xhr-streaming, iframe-eventsource
│ └─ Polling fallback: xhr-polling (last resort)
├─ Allowed Origins: https://gameverse.gg, http://localhost:5173
└─ Heartbeat: Server sends every 10s, expects client every 10s

MESSAGE BROKER CONFIGURATION:

Simple Broker Prefixes: /topic, /queue

Application Destination Prefix: /app

User Destination Prefix: /user

DESTINATION TYPES:

- └─ /topic/... → Broadcast to all subscribers (tournament-wide)
- └─ /queue/... → Point-to-point (used with /user prefix)
- └─ /app/... → Client-to-server messages (@MessageMapping)

USER DESTINATIONS:

- /user/{userId}/queue/notifications → Personal notifications
- /user/{userId}/queue/credentials → Credential delivery (private)
- /user/{userId}/queue/team → Team-specific updates

WHY /user/... DESTINATIONS ARE SECURE:

Spring STOMP resolves /user/{userId} to the actual connected session.
A user cannot subscribe to /user/other-user-id/queue/credentials.
Spring Security enforces this at the STOMP handshake level.

3.2 Connection Endpoints

WEBSOCKET URLs:

Production:

- └─ Primary: wss://api.gameverse.gg/ws
- └─ SockJS: https://api.gameverse.gg/ws (SockJS negotiation)

Staging:

- └─ Primary: wss://api-staging.gameverse.gg/ws
- └─ SockJS: https://api-staging.gameverse.gg/ws

Local Development:

- └─ Primary: ws://localhost:8080/ws
- └─ SockJS: http://localhost:8080/ws

OBS Browser Source:

- └─ Uses SockJS URL (https) with polling fallback
 - OBS Chromium supports WebSocket, SockJS preferred for reliability

SECTION 4 — COMPLETE EVENT CATALOG

4.1 Server → Client Events (Broadcast)

COMPLETE EVENT CATALOG – SERVER → CLIENT

GROUP 1: TOURNAMENT LIFECYCLE EVENTS

Event Code	Destination	Audience
TOURNAMENT_STATUS_CHANGED	/topic/t.{id}.status	All
TOURNAMENT_COMPLETED	/topic/t.{id}.status	All
TOURNAMENT_CANCELLED	/topic/t.{id}.status	All
ANNOUNCEMENT_PUBLISHED	/topic/t.{id}.announce	Participants

GROUP 2: REGISTRATION & CHECK-IN EVENTS

Event Code	Destination	Audience
REGISTRATION_STATUS_CHANGED	/user/{id}/queue/team	Captain only
CHECKIN_UPDATED	/topic/t.{id}.checkins	Staff only
CHECKIN_GRID_REFRESH	/topic/t.{id}.checkins	Staff only
WAITLIST_SLOT_OFFERED	/user/{id}/queue/team	Captain only

GROUP 3: MATCH LIFECYCLE EVENTS

Event Code	Destination	Audience
MATCH_STATUS_CHANGED	/topic/t.{id}.matches	All
MATCH_LOBBY_OPENED	/topic/t.{id}.matches	All
MATCH_STARTED	/topic/t.{id}.matches	All
MATCH_PAUSED	/topic/t.{id}.matches	All
MATCH_RESUMED	/topic/t.{id}.matches	All
MATCH_VOIDED	/topic/t.{id}.matches	All
MATCH_COMPLETED	/topic/t.{id}.matches	All

GROUP 4: SCORING & RESULTS EVENTS

Event Code	Destination	Audience
RESULT_SUBMITTED	/topic/t.{id}.command	Staff only
RESULT_PUBLISHED	/topic/t.{id}.matches	All
RESULT_CORRECTION_APPLIED	/topic/t.{id}.matches	All
SCORE_ANOMALY_FLAGGED	/topic/t.{id}.command	Staff only

GROUP 5: LEADERBOARD EVENTS

Event Code	Destination	Audience
LEADERBOARD_UPDATED	/topic/t.{id}.leaderboard	All + OBS
LEADERBOARD_LOCKED	/topic/t.{id}.leaderboard	All

GROUP 6: CREDENTIAL EVENTS

Event Code	Destination	Audience
CREDENTIALS_RELEASED	/user/{id}/queue/creds	Match players
CREDENTIALS_UPDATED	/user/{id}/queue/creds	Match players
CREDENTIALS_REVOKED	/user/{id}/queue/creds	Match players
CREDENTIALS_EXPIRING_SOON	/user/{id}/queue/creds	Match players

GROUP 7: DISPUTE EVENTS

Event Code	Destination	Audience
------------	-------------	----------

DISPUTE_CREATED	/topic/t.{id}.command	Staff only
DISPUTE_RESOLVED	/topic/t.{id}.command	Staff only
DISPUTE_RESOLVED_NOTIFY	/user/{id}/queue/notifs	Captain only
DISPUTE_ESCALATED	/topic/t.{id}.command	Staff only

GROUP 8: BROADCAST & OVERLAY EVENTS

Event Code	Destination	Audience
STREAM_HEALTH_UPDATE	/topic/t.{id}.stream	Producer+Dir
STREAM_STARTED	/topic/t.{id}.stream	All
STREAM_ENDED	/topic/t.{id}.stream	All
OVERLAY_COMMAND	/topic/t.{id}.overlays	OBS sources
OVERLAY_DATA_REFRESH	/topic/t.{id}.overlays	OBS sources

GROUP 9: COMMAND CENTER EVENTS

Event Code	Destination	Audience
PENDING_ACTIONS_UPDATED	/topic/t.{id}.command	Staff only
STAFF_ALERT	/topic/t.{id}.command	Staff only
DQ_RECOMMENDED	/topic/t.{id}.command	Director only
DQ_CONFIRMED	/topic/t.{id}.matches	All

GROUP 10: PERSONAL NOTIFICATION EVENTS

Event Code	Destination	Audience
NOTIFICATION_RECEIVED	/user/{id}/queue/notifs	1 user
UNREAD_COUNT_UPDATED	/user/{id}/queue/notifs	1 user
PRIZE_PAYOUT_READY	/user/{id}/queue/notifs	1 captain
PRIZE_PAYOUT_COMPLETED	/user/{id}/queue/notifs	1 captain

4.2 Client → Server Events

CLIENT → SERVER MESSAGE MAPPINGS (@MessageMapping):

/app/tournament.subscribe

- └ Purpose: Client declares active tournament subscription
- └ Triggers: Server sends current state snapshot
- └ Handler: TournamentSubscriptionController.subscribe()

/app/tournament.unsubscribe

- └ Purpose: Client leaving tournament context
- └ Handler: TournamentSubscriptionController.unsubscribe()

/app/chat.send

- └ Purpose: Send chat message to tournament chat
- └ Handler: ChatController.sendMessage()

/app/overlay.heartbeat

- └ Purpose: OBS overlay pings to confirm connection alive

└─ Handler: OverlayController.heartbeat()

/app/credentials.acknowledge

└─ Purpose: Player confirms credentials received and viewed

└─ Handler: CredentialController.acknowledge()

/app/sync.request

└─ Purpose: Client requests full state after reconnect

└─ Handler: SyncController.requestFullState()

SECTION 5 — EVENT PAYLOAD SPECIFICATIONS

5.1 Universal Event Envelope

ALL STOMP MESSAGES USE THIS ENVELOPE:

```
{
  "eventCode": "LEADERBOARD_UPDATED",
  "eventId": "evt-uuid-abc123",
  "tournamentId": "tournament-uuid",
  "timestamp": "2025-06-15T14:45:00.123Z",
  "sequenceNumber": 47,
  "data": { ... event-specific payload ... },
  "meta": {
    "triggeredBy": "match_result_published",
    "matchId": "match-uuid",
    "version": "1.0"
  }
}
```

FIELDS EXPLAINED:

└─ eventCode: Unique string identifying event type
└─ eventId: UUID for deduplication on reconnect
└─ tournamentId: Always included (context)
└─ timestamp: ISO-8601 UTC (client uses for display)
└─ sequenceNumber: Monotonically increasing per tournament
└─ data: Event-specific payload (defined below)
└─ meta: Context about what triggered this event
└─ → Client detects missed events if gap detected

5.2 LEADERBOARD_UPDATED Payload

```
{
  "eventCode": "LEADERBOARD_UPDATED",
  "eventId": "evt-uuid-001",
  "tournamentId": "tid-001",
  "timestamp": "2025-06-15T14:45:00.123Z",
  "sequenceNumber": 47,
```

```
"data": {
  "triggeredByMatch": {
    "matchId": "match-uuid",
    "matchNumber": 3,
    "roundNumber": 2
  },
  "matchesCompleted": 8,
  "totalMatches": 16,
  "isLocked": false,
  "leaderboard": [
    {
      "rank": 1,
      "previousRank": 2,
      "rankChange": 1,
      "rankChangeDirection": "UP",
      "registrationId": "reg-uuid",
      "teamId": "team-uuid",
      "teamName": "Storm Squad",
      "teamTag": "STM",
      "logoUrl": "https://cdn.gameverse.gg/teams/stm.webp",
      "totalPoints": 62.0,
      "totalKills": 38,
      "chickenDinners": 2,
      "matchesPlayed": 8,
      "avgPlacement": 3.2,
      "matchBreakdown": [
        {
          "matchNumber": 1,
          "placement": 1,
          "kills": 8,
          "points": 23.0,
          "isChickenDinner": true
        },
        {
          "matchNumber": 2,
          "placement": 4,
          "kills": 5,
          "points": 13.0,
          "isChickenDinner": false
        }
      ],
      "isEliminated": false,
      "isDisqualified": false
    },
    {
      "rank": 2,
      "previousRank": 1,
      "rankChange": -1,
      "rankChangeDirection": "DOWN",
      "registrationId": "reg-uuid-2",
      "teamId": "team-uuid-2",
      "teamName": "Hydra Esports",
      "teamTag": "HYD",
      "logoUrl": "https://cdn.gameverse.gg/teams/hyd.webp",
      "totalPoints": 58.0,
      "totalKills": 31,
```

```

        "chickenDinners": 1,
        "matchesPlayed": 8,
        "avgPlacement": 3.9,
        "isEliminated": false,
        "isDisqualified": false
    }
],
"lastUpdatedAt": "2025-06-15T14:45:00.123Z"
},
"meta": {
    "triggeredBy": "match_result_published",
    "matchId": "match-uuid",
    "version": "1.0"
}
}

```

5.3 CREDENTIALS_RELEASED Payload

```

{
    "eventCode": "CREDENTIALS_RELEASED",
    "eventId": "evt-uuid-cred-001",
    "tournamentId": "tid-001",
    "timestamp": "2025-06-15T14:25:00.456Z",
    "sequenceNumber": 31,
    "data": {
        "matchId": "match-uuid",
        "matchNumber": 3,
        "roundNumber": 2,
        "matchLabel": "Round 2 – Match 3",
        "scheduledStart": "2025-06-15T14:30:00Z",
        "credential": {
            "credentialId": "cred-uuid",
            "roomId": "A7B3C9",
            "password": "XK29",
            "slotNumber": 7,
            "releasedAt": "2025-06-15T14:25:00Z",
            "expiresAt": "2025-06-15T14:55:00Z",
            "expiresInSeconds": 1800
        },
        "instructions": "Open BGMI → Custom Room → Enter Room ID → Join Slot 7",
        "urgency": "HIGH"
    },
    "meta": {
        "triggeredBy": "credential_manual_release",
        "releasedBy": "referee-user-uuid",
        "version": "1.0"
    }
}

```

CRITICAL SECURITY NOTE:

CREDENTIALS_RELEASED is NEVER published to a /topic/ destination.

It is ONLY sent to:

/user/{specific-player-id}/queue/credentials

This means Spring STOMP delivers it ONLY to the authenticated WebSocket session of that exact user. No other subscriber can receive this message, even if they know the match ID.

The roomId and password fields are decrypted server-side immediately before SimpMessagingTemplate.convertAndSendToUser() is called. Encrypted values never appear in WebSocket frames.

5.4 MATCH_STATUS_CHANGED Payload

```
{
  "eventCode": "MATCH_STATUS_CHANGED",
  "eventId": "evt-uuid-match-001",
  "tournamentId": "tid-001",
  "timestamp": "2025-06-15T14:30:05.789Z",
  "sequenceNumber": 33,
  "data": {
    "matchId": "match-uuid",
    "matchNumber": 3,
    "roundNumber": 2,
    "matchLabel": "Round 2 – Match 3",
    "previousStatus": "LOBBY_OPEN",
    "currentStatus": "IN_PROGRESS",
    "actualStartTime": "2025-06-15T14:30:05Z",
    "estimatedEndTime": "2025-06-15T15:05:00Z",
    "teamsInMatch": 16,
    "assignedReferee": {
      "userId": "ref-uuid",
      "username": "RefMike"
    }
  },
  "meta": {
    "triggeredBy": "referee_action",
    "actorUserId": "ref-uuid",
    "version": "1.0"
  }
}
```

5.5 MATCH_PAUSED Payload

```
{
  "eventCode": "MATCH_PAUSED",
  "eventId": "evt-uuid-pause-001",
  "tournamentId": "tid-001",
```

```

"timestamp": "2025-06-15T14:48:00.000Z",
"sequenceNumber": 38,
"data": {
  "matchId": "match-uuid",
  "matchNumber": 3,
  "roundNumber": 2,
  "pauseId": "pause-uuid",
  "reason": "GAME_CRASH",
  "reasonLabel": "Game Server Crash",
  "reasonNotes": "Server crashed in Zone 3. Investigating.",
  "pausedAt": "2025-06-15T14:48:00Z",
  "estimatedResumeAt": "2025-06-15T15:00:00Z",
  "estimatedWaitMinutes": 12,
  "message": "Match 3 is paused due to a game crash. Please standby.",
  "urgency": "HIGH"
},
"meta": {
  "triggeredBy": "referee_action",
  "actorUserId": "ref-uuid",
  "version": "1.0"
}
}

```

5.6 RESULT_PUBLISHED Payload

```

{
  "eventCode": "RESULT_PUBLISHED",
  "eventId": "evt-uuid-result-001",
  "tournamentId": "tid-001",
  "timestamp": "2025-06-15T12:45:00.000Z",
  "sequenceNumber": 22,
  "data": {
    "matchId": "match-uuid",
    "matchNumber": 1,
    "roundNumber": 1,
    "results": [
      {
        "placement": 1,
        "teamId": "team-uuid-stm",
        "teamName": "Storm Squad",
        "teamTag": "STM",
        "rawKills": 8,
        "effectiveKills": 6,
        "killPoints": 6.0,
        "placementPoints": 15.0,
        "totalPoints": 21.0,
        "isChickenDinner": true,
        "isDisqualified": false
      },
      {
        "placement": 2,
        "teamId": "team-uuid-hyd",
        "teamName": "Hydra Esports",

```

```

        "teamTag": "HYD",
        "rawKills": 6,
        "effectiveKills": 6,
        "killPoints": 6.0,
        "placementPoints": 12.0,
        "totalPoints": 18.0,
        "isChickenDinner": false,
        "isDisqualified": false
    }
],
"disputeWindowClosesAt": "2025-06-15T13:15:00Z",
"disputeWindowMinutes": 30,
"screenshotUrl": "https://cdn.gameverse.gg/evidence/result-m1.webp"
},
"meta": {
    "triggeredBy": "director_verification",
    "verifiedBy": "director-user-uuid",
    "version": "1.0"
}
}

```

5.7 CREDENTIALS_REVOKED Payload

```

{
    "eventCode": "CREDENTIALS_REVOKED",
    "eventId": "evt-uuid-revoke-001",
    "tournamentId": "tid-001",
    "timestamp": "2025-06-15T14:31:00.000Z",
    "sequenceNumber": 34,
    "data": {
        "matchId": "match-uuid",
        "matchNumber": 3,
        "reason": "SECURITY_ROTATION",
        "message": "Room credentials have been updated. New credentials arriving shortly.",
        "revokedCredentialId": "cred-uuid-old",
        "newCredentialsIncoming": true,
        "urgency": "CRITICAL"
    },
    "meta": {
        "triggeredBy": "security_rotation",
        "actorUserId": "ref-uuid",
        "version": "1.0"
    }
}

```

5.8 TOURNAMENT_STATUS_CHANGED Payload

```

{
    "eventCode": "TOURNAMENT_STATUS_CHANGED",
    "eventId": "evt-uuid-tstatus-001",

```

```

"tournamentId": "tid-001",
"timestamp": "2025-06-15T10:00:00.000Z",
"sequenceNumber": 1,
"data": {
  "previousStatus": "CHECK_IN",
  "currentStatus": "LIVE",
  "tournamentName": "BGMI Weekend Cup #12",
  "message": "Tournament is now LIVE! Good luck to all teams.",
  "currentRound": 1,
  "totalRounds": 4,
  "firstMatchStartsAt": "2025-06-15T12:00:00Z"
},
"meta": {
  "triggeredBy": "director_action",
  "actorUserId": "director-uuid",
  "version": "1.0"
}
}

```

5.9 PENDING_ACTIONS_UPDATED Payload (Command Center)

```

{
  "eventCode": "PENDING_ACTIONS_UPDATED",
  "eventId": "evt-uuid-cc-001",
  "tournamentId": "tid-001",
  "timestamp": "2025-06-15T14:45:05.000Z",
  "sequenceNumber": 48,
  "data": {
    "summary": {
      "criticalCount": 1,
      "highCount": 2,
      "mediumCount": 0,
      "totalCount": 3
    },
    "actions": [
      {
        "actionId": "action-001",
        "type": "DISPUTE_OPEN",
        "priority": "CRITICAL",
        "title": "Dispute: Incorrect Kill Count",
        "description": "Hydra Esports disputes kill count in Match 3",
        "referenceId": "DSP-BGMI-0042",
        "createdAt": "2025-06-15T14:44:30Z",
        "windowClosesAt": "2025-06-15T15:14:30Z",
        "actionUrl": "/command-center/tid-001/disputes/DSP-BGMI-0042"
      },
      {
        "actionId": "action-002",
        "type": "RESULT_PENDING_VERIFICATION",
        "priority": "HIGH",
        "title": "Results awaiting verification: Match 4",
        "description": "Referee submitted results 5 minutes ago",
        "referenceId": "match-uuid-4",

```

```

    "createdAt": "2025-06-15T14:40:00Z",
    "actionUrl": "/command-center/tid-001/scoring/match-uuid-4"
  },
  {
    "actionId": "action-003",
    "type": "RESULT_PENDING_VERIFICATION",
    "priority": "HIGH",
    "title": "Results awaiting verification: Match 5",
    "referenceId": "match-uuid-5",
    "createdAt": "2025-06-15T14:42:00Z",
    "actionUrl": "/command-center/tid-001/scoring/match-uuid-5"
  }
],
"upcomingMatch": {
  "matchId": "match-uuid-6",
  "matchNumber": 6,
  "scheduledStart": "2025-06-15T15:00:00Z",
  "minutesUntilStart": 14
}
},
"meta": {
  "triggeredBy": "dispute_created",
  "version": "1.0"
}
}

```

5.10 OVERLAY_COMMAND Payload

```

{
  "eventCode": "OVERLAY_COMMAND",
  "eventId": "evt-uuid-ovl-001",
  "tournamentId": "tid-001",
  "timestamp": "2025-06-15T14:45:10.000Z",
  "sequenceNumber": 49,
  "data": {
    "command": "SHOW",
    "overlayType": "RESULT_SPLASH",
    "overlayId": "ovl-uuid-result",
    "params": {
      "matchNumber": 3,
      "results": [
        { "placement": 1, "teamName": "Storm Squad", "points": 21.0 },
        { "placement": 2, "teamName": "Hydra Esports", "points": 18.0 },
        { "placement": 3, "teamName": "Phoenix Rising", "points": 15.0 }
      ],
      "displayDurationSeconds": 15,
      "animation": "SLIDE_IN"
    }
  },
  "meta": {
    "triggeredBy": "result_published",
    "version": "1.0"
  }
}

```

```
}
```

OVERLAY COMMANDS:

```
|— SHOW           → Make overlay visible
|— HIDE           → Make overlay invisible
|— UPDATE         → Update data without visibility change
|— REFRESH        → Force full data reload
|— SCENE_CHANGE   → OBS should switch to named scene
```

5.11 CHECKIN_UPDATED Payload

```
{
  "eventCode": "CHECKIN_UPDATED",
  "eventId": "evt-uuid-ci-001",
  "tournamentId": "tid-001",
  "timestamp": "2025-06-15T10:23:45.000Z",
  "sequenceNumber": 5,
  "data": {
    "summary": {
      "totalApproved": 64,
      "totalCheckedIn": 58,
      "totalNotCheckedIn": 6,
      "totalNoShow": 0,
      "checkinWindowClosesAt": "2025-06-15T11:30:00Z",
      "minutesRemaining": 66
    },
    "updatedTeam": {
      "registrationId": "reg-uuid",
      "slotNumber": 7,
      "teamName": "Hydra Esports",
      "teamTag": "HYD",
      "checkedIn": true,
      "checkinAt": "2025-06-15T10:23:45Z",
      "checkinType": "SELF"
    },
    "notCheckedIn": [
      {
        "registrationId": "reg-uuid-2",
        "slotNumber": 12,
        "teamName": "Phoenix Rising",
        "teamTag": "PHX"
      }
    ]
  },
  "meta": {
    "triggeredBy": "team_self_checkin",
    "version": "1.0"
  }
}
```

5.12 STREAM_HEALTH_UPDATE Payload

```
{
  "eventCode": "STREAM_HEALTH_UPDATE",
  "eventId": "evt-uuid-stream-001",
  "tournamentId": "tid-001",
  "timestamp": "2025-06-15T14:45:00.000Z",
  "sequenceNumber": 46,
  "data": {
    "isLive": true,
    "streamStartedAt": "2025-06-15T10:00:00Z",
    "streamDurationMinutes": 285,
    "currentViewerCount": 1204,
    "peakViewerCount": 1842,
    "avgViewerCount": 987,
    "quality": {
      "bitrateKbps": 6000,
      "fps": 60,
      "droppedFramesPct": 0.02,
      "cpuUsagePct": 47,
      "healthScore": "EXCELLENT"
    },
    "obsConnected": true,
    "currentScene": "Match Live",
    "platform": "YOUTUBE",
    "streamUrl": "https://youtube.com/live/abc123"
  },
  "meta": {
    "triggeredBy": "scheduled_health_poll",
    "version": "1.0"
  }
}
```

5.13 DISPUTE_CREATED Payload (Staff Only)

```
{
  "eventCode": "DISPUTE_CREATED",
  "eventId": "evt-uuid-dsp-001",
  "tournamentId": "tid-001",
  "timestamp": "2025-06-15T14:44:30.000Z",
  "sequenceNumber": 45,
  "data": {
    "disputeId": "dsp-uuid",
    "referenceNumber": "DSP-BGMI-0042",
    "matchId": "match-uuid",
    "matchNumber": 3,
    "team": {
      "teamId": "team-uuid-hyd",
      "teamName": "Hydra Esports",
      "teamTag": "HYD"
    },
    "disputeType": "INCORRECT_KILL_COUNT",
  },
}
```

```

    "description": "Team had 9 kills. System recorded 7.",
    "claimedKills": 9,
    "recordedKills": 7,
    "priority": "HIGH",
    "affectsStandings": true,
    "standingImpactPoints": 2.0,
    "windowClosesAt": "2025-06-15T15:14:30Z",
    "windowMinutesRemaining": 30,
    "actionRequired": true
  },
  "meta": {
    "triggeredBy": "captain_dispute_submission",
    "submittedByUserId": "captain-uuid",
    "version": "1.0"
  }
}

```

5.14 NOTIFICATION_RECEIVED Payload

```

{
  "eventCode": "NOTIFICATION_RECEIVED",
  "eventId": "evt-uuid-notif-001",
  "tournamentId": "tid-001",
  "timestamp": "2025-06-15T14:25:00.000Z",
  "sequenceNumber": 30,
  "data": {
    "notificationId": "notif-uuid",
    "type": "CREDENTIALS_AVAILABLE",
    "priority": "CRITICAL",
    "title": "🔑 Room credentials available for Match 3!",
    "body": "Open BGMI and join Custom Room A7B3C9. Your slot: 7",
    "actionUrl": "/tournaments/tid-001/my-matches",
    "actionLabel": "View Credentials",
    "iconType": "KEY",
    "expiresAt": "2025-06-15T14:55:00Z",
    "isRead": false,
    "createdAt": "2025-06-15T14:25:00Z",
    "newUnreadCount": 3
  },
  "meta": {
    "triggeredBy": "credential_released",
    "version": "1.0"
  }
}

```

SECTION 6 — THE COMPLETE REAL-TIME PIPELINE

6.1 Pipeline: Score Update → OBS Overlay (Detailed)

COMPLETE SCORE-TO-OVERLAY PIPELINE

STAGE 1: INPUT CAPTURE

Referee opens Match Control Panel in Command Center

Match status: IN_PROGRESS

Referee sees result entry form (16 rows × placement + kills)

Referee enters: Placement 1-16, Kills 0-99 per team

Client-side validation (React Hook Form + Zod):

- └─ All 16 placements entered (no blanks)
- └─ Placements 1-16 all unique (set validation)
- └─ Kills ≥ 0 for all teams
- └─ Screenshot uploaded (required)
- └─ Validation passes → form enabled for submission

Referee clicks "Submit Results"

STAGE 2: NETWORK TRANSMISSION

POST /api/v1/tournaments/{id}/matches/{id}/results

Headers:

- └─ Authorization: Bearer {jwt}
- └─ Content-Type: application/json
- └─ X-Request-ID: req-uuid-abc

Body: { screenshot_url, results: [{ registration_id, placement, raw_kills }, × 16] }

STAGE 3: BACKEND VALIDATION (Spring Boot)

Spring Security:

- └─ JWT signature valid? (RS256 public key)
- └─ Token not expired?
- └─ @PreAuthorize: User is REFEREE of this match?
OR TOURNAMENT_DIRECTOR of this tournament?

Bean Validation (@Valid):

- └─ results list: not empty, size = teams_per_match
- └─ Each placement: 1 to teams_per_match
- └─ Each raw_kills: ≥ 0
- └─ screenshot_url: valid URL format
- └─ All placement values unique (custom validator)

Business Rule Validation (Service layer):

- └─ Match exists for this tournament?
- └─ Match status is IN_PROGRESS? (not PAUSED, COMPLETED, etc)
- └─ No result already submitted for this match?

- └─ All registration_ids belong to match slots?
- └─ Tournament scoring template loaded correctly?

STAGE 4: DATABASE TRANSACTION

@Transactional begins (InnoDB transaction):

4a. MatchResult entity created:

```
INSERT INTO match_results (match_id, submitted_by,
    screenshot_url, submission_status, submitted_at)
```

4b. ScoringEngine.calculate() executes for each team:

```
effective_kills = MIN(raw_kills, kill_cap)
kill_points = effective_kills × kill_pts_each
placement_points = placement_points_map[placement]
total_points = placement_points + kill_points
is_chicken_dinner = (placement == 1)
```

4c. TeamMatchResult entities created (16 rows):

```
INSERT INTO team_match_results (result_id, match_id,
    registration_id, placement, raw_kills, effective_kills,
    kill_points, placement_points, total_points,
    is_chicken_dinner) VALUES (× 16)
```

4d. Match status updated:

```
UPDATE matches SET status = 'RESULT_SUBMITTED'
    (or 'PENDING_VERIFICATION' if director verify mode)
```

4e. Anomaly detection runs:

- └─ Any team has kills > kill_cap × 2? (flag)
- └─ Any team had 0 placement but non-zero kills? (flag)

@Transactional COMMITS to MySQL

– Database is now consistent –

STAGE 5: DOMAIN EVENT DISPATCH

Spring ApplicationEventPublisher fires AFTER commit:

@TransactionalEventListener(phase = AFTER_COMMIT)

If AUTO_PUBLISH mode:

```
→ MatchResultPublishedEvent { matchId, tournamentId,
    resultId, triggeredBy }
```

If DIRECTOR_VERIFY mode:

```
→ MatchResultSubmittedEvent (pending, not yet published)
→ Director must verify → then triggers Published event
```

STAGE 6: LEADERBOARD RECALCULATION

LeaderboardEventListener.onMatchResultPublished():

@Async("leaderboardExecutor")

6a. Aggregate query on MySQL:

```
SELECT registration_id,  
       SUM(total_points) as total_pts,  
       SUM(raw_kills) as total_kills,  
       SUM(is_chicken_dinner) as chicken_dinners,  
       COUNT(*) as matches_played,  
       AVG(placement) as avg_placement  
FROM team_match_results tmr  
JOIN match_results mr ON tmr.result_id = mr.id  
WHERE tournament_id = :tid  
AND mr.submission_status = 'PUBLISHED'  
GROUP BY registration_id  
ORDER BY total_pts DESC,  
         chicken_dinners DESC,  
         total_kills DESC
```

6b. Rank numbers assigned:

Iterate result list: rank = index + 1
Handle ties: same rank if all tiebreaker fields equal

6c. Rank changes calculated:

For each team: rankChange = previousRank - currentRank
Store previousRank before overwriting

6d. Batch UPDATE leaderboard_entries:

```
UPDATE leaderboard_entries SET  
    current_rank = ?, previous_rank = ?,  
    rank_change = ?, total_points = ?,  
    total_kills = ?, chicken_dinners = ?,  
    last_updated_at = NOW()  
WHERE tournament_id = ? AND registration_id = ?  
(executed as batch for all 64 teams)
```

6e. Cache invalidated:

Caffeine cache evict: leaderboard::{tournamentId}
Cache refreshed with new leaderboard data

6f. LeaderboardSnapshot saved (historical record):

```
INSERT INTO leaderboard_snapshots (tournament_id,  
    triggered_by_match, snapshot_data, snapshot_at)
```

STAGE 7: WEBSOCKET BROADCAST

WebSocketEventPublisher.broadcastLeaderboard():

7a. Build LEADERBOARD_UPDATED payload:

- └ Full leaderboard array (all 64 teams)
- └ Rank changes (UP/DOWN/SAME per team)
- └ Match breakdown per team
- └ Tournament progress (8/16 matches)
- └ Sequence number incremented

7b. SimpMessagingTemplate.convertAndSend():

Destination: /topic/tournament.{id}.leaderboard
Payload: LEADERBOARD_UPDATED JSON

```

7c. Simultaneously publish RESULT_PUBLISHED:
  Destination: /topic/tournament.{id}.matches
  Payload: RESULT_PUBLISHED JSON (match result data)

7d. Update Command Center:
  Destination: /topic/tournament.{id}.command
  Payload: PENDING_ACTIONS_UPDATED

7e. Trigger OBS Overlay Command:
  Destination: /topic/tournament.{id}.overlays
  Payload: OVERLAY_COMMAND { command: SHOW,
    overlayType: RESULT_SPLASH, params: top3 }

```

STAGE 8: DELIVERY TO CONSUMERS

STOMP Broker routes messages to all subscribers:

```

Consumer A: Public Website (React)
├─ useStompSubscription("/topic/tournament.{id}.leaderboard")
├─ Receives LEADERBOARD_UPDATED
├─ Zustand store: setLeaderboard(event.data.leaderboard)
├─ React re-renders <LeaderboardTable />
└─ CSS: rows animate to new positions (600ms transition)

```

```

Consumer B: Player Dashboard (React)
├─ Same LEADERBOARD_UPDATED event
├─ Filter: find own team by registrationId
└─ Update "Your Standing: 2nd (58pts)" display

```

```

Consumer C: Command Center (React)
├─ Leaderboard section updates
├─ RESULT_PUBLISHED → pending verification badge clears
└─ PENDING_ACTIONS_UPDATED → action list refreshes

```

```

Consumer D: OBS Browser Source (Vanilla JS + STOMP)
├─ Receives LEADERBOARD_UPDATED
├─ updateLeaderboardDOM(event.data.leaderboard)
├─ Animate rank changes with CSS transitions
├─ Receives OVERLAY_COMMAND { SHOW RESULT_SPLASH }
└─ showResultSplash(top3) → 15s display → auto-hide

```

```

TOTAL PIPELINE DURATION: ~200-500ms (MVP, single instance)
TARGET: < 3000ms  ✓

```

6.2 Credential Release Pipeline

CREDENTIAL RELEASE PIPELINE (highest security path):

```

T+0      Referee enters Room ID + Password in Match Control Panel
         POST /api/v1/matches/{id}/credentials
         { roomId, password, releaseMode: "TIMED",

```

```
scheduledReleaseAt: "2025-06-15T14:25:00Z" }
```

T+50ms Spring Security: RBAC check (is referee of this match)
AES-256-GCM encrypt: roomId + password
Store encrypted values in room_credentials table
Store release schedule in scheduled_releases table
Audit log: credential_entered

T+0 (Time advances to scheduled release time: 14:25:00)

T+0 Spring @Scheduled (every 60s) detects:
SELECT * FROM room_credentials
WHERE scheduled_release_at <= NOW()
AND is_released = false
→ Found: match-uuid credential

T+20ms CredentialReleaseService.release(matchId):

Step 1: Determine recipients:
SELECT DISTINCT user_id FROM registration_rosters rr
JOIN tournament_registrations tr ON rr.registration_id = tr.id
JOIN match_slots ms ON ms.registration_id = tr.id
WHERE ms.match_id = :matchId
AND tr.status = 'APPROVED'
AND rr.is_active = true
→ Result: 64 user_ids

Step 2: Decrypt credentials (server-side only):
AES-256-GCM decrypt: roomId_encrypted → "A7B3C9"
AES-256-GCM decrypt: password_encrypted → "XK29"
(Decrypted values exist in JVM memory ONLY during delivery)

Step 3: For each user, determine slot number:
SELECT slot_number FROM match_slots ms
JOIN tournament_registrations tr ON ms.registration_id = tr.id
JOIN registration_rosters rr ON rr.registration_id = tr.id
WHERE ms.match_id = :matchId AND rr.user_id = :userId

Step 4: Build personalized payload per user:
{
 eventCode: "CREDENTIALS_RELEASED",
 data: {
 roomId: "A7B3C9", ← PLAINTEXT, only in this delivery
 password: "XK29", ← PLAINTEXT, only in this delivery
 slotNumber: 7, ← Personalized per user
 expiresAt: "14:55",
 ...
 }
}

T+80ms SimpMessagingTemplate.convertAndSendToUser():
For each of 64 users:
destination = "/user/{userId}/queue/credentials"
payload = personalized credential payload

Spring STOMP routes ONLY to that user's WebSocket session.

Other users CANNOT subscribe to another user's queue.

T+90ms Notification dispatch (@Async, parallel):
└─ In-app: Already delivered via STOMP above
└─ Push: FCM/APNs notification (title: "🔑 Credentials ready!")
└─ SMS: MSG91 → "Room credentials for Match 3. Open GameVerse. -GVRSE"

T+100ms Audit log batch insert:
INSERT INTO credential_logs (credential_id, user_id,
action = 'RELEASED', created_at) VALUES x 64

T+150ms Player's screen:
└─ STOMP message arrives on /user/{id}/queue/credentials
└─ React hook fires: onCredentialReceived(payload)
└─ Credential Card renders: Room ID A7B3C9, Password XK29
└─ 30-minute countdown timer starts
└─ Lobby Readiness: "Not Yet Acknowledged" → Player taps "Got It"

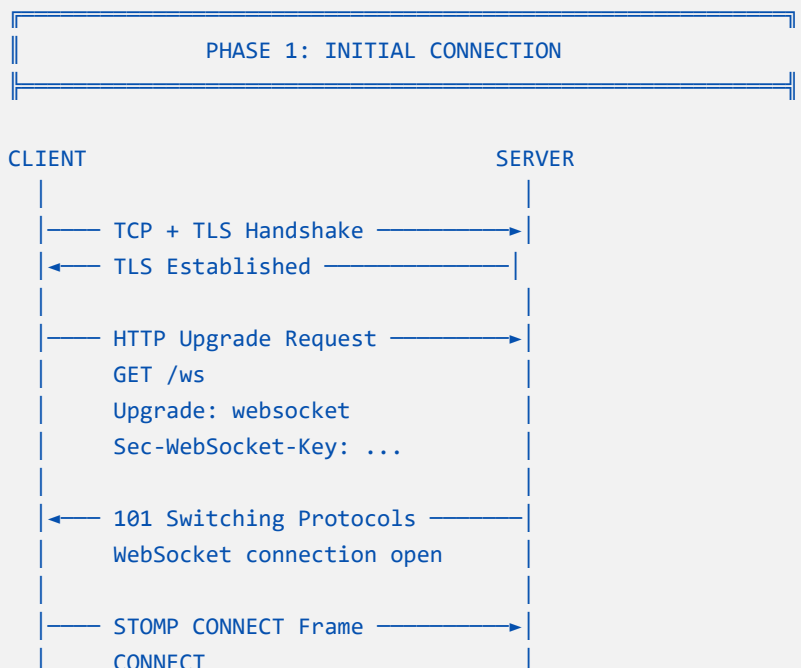
T+160ms Player taps "Got It":
POST /api/v1/matches/{matchId}/credentials/acknowledge
→ credential_logs: action = 'ACKNOWLEDGED'
→ Referee's Lobby Readiness Tracker: Hydra Esports ✅

TOTAL: Credentials visible on player screen in ~150ms of release

SECTION 7 — CONNECTION LIFECYCLE & AUTHENTICATION

7.1 Full Connection Lifecycle

STOMP CONNECTION LIFECYCLE:



```
accept-version:1.2
heart-beat:10000,10000
Authorization:Bearer {jwt}    ← JWT in STOMP header
X-Tournament-ID:{id}         ← Optional context hint
\0
```

Spring ChannelInterceptor:

1. Extract Authorization header
2. Validate JWT (RS256)
3. Check expiry
4. Build GameVerseAuthentication
5. Store in WebSocket session attributes

← STOMP CONNECTED Frame

```
CONNECTED
version:1.2
heart-beat:10000,10000
session-id:{ws-session-uuid}
user-name:{username}
\0
```

PHASE 2: SUBSCRIPTION SETUP

→ STOMP SUBSCRIBE Frame

```
SUBSCRIBE
id:sub-0
destination:/topic/tournament
.{id}.leaderboard
\0
```

Spring validates subscription:

- /topic/tournament.{id}.* → Public: OK
- /topic/tournament.{id}.command → Staff check
- /user/{id}/queue/* → Own ID check

← Initial State Sync

```
MESSAGE (LEADERBOARD_UPDATED)
Current leaderboard snapshot
(sent immediately on sub)
```

PHASE 3: ACTIVE SESSION

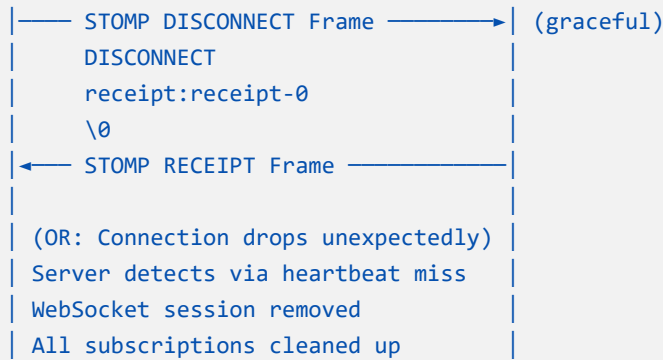
← Live Events (pushed as they occur)

→ STOMP HEARTBEAT (every 10s)

← STOMP HEARTBEAT (every 10s)

(keepalive, no payload)

PHASE 4: DISCONNECTION



7.2 Token Expiry Handling During Active Session

JWT EXPIRY MANAGEMENT IN WEBSOCKET SESSION:

Problem:

Access tokens expire after 15 minutes.
WebSocket sessions last 6-12 hours during tournaments.

Solution: Token refresh before WebSocket re-authentication

TIMELINE:

0:00 User connects with valid JWT (exp: +15min)
14:00 React detects token about to expire:
Axios interceptor: check token expiry on each REST call
If $\text{exp} - \text{now} < 60$ seconds $\rightarrow$ refresh proactively
14:01 POST /api/v1/auth/token/refresh
Cookie: refresh_token=...
Response: { accessToken: "new-jwt..." }
14:01 New access token stored in React memory (Zustand authStore)
14:01 STOMP client updated:
client.configure({
 connectHeaders: {
 Authorization: `Bearer \${newAccessToken}`
 }
})

NOTE: Existing WebSocket connection is NOT dropped.
Spring stores user authentication in WebSocket session attributes.
The session remains authenticated for the session lifetime.

Only on RECONNECT does the new token matter.
Token refresh ensures reconnects succeed without re-login.

SECTION 8 — RECONNECTION BEHAVIOR & RECOVERY

8.1 Reconnection Strategy (React Client)

RECONNECTION CONFIGURATION (@stomp/stompjs):

```
const stompClient = new Client({
  brokerURL: 'wss://api.gameverse.gg/ws',
  websocketFactory: () => new SockJS('https://api.gameverse.gg/ws'),

  reconnectDelay: 5000,           // Initial reconnect: 5 seconds
  heartbeatIncoming: 10000,       // Expect server heartbeat every 10s
  heartbeatOutgoing: 10000,       // Send heartbeat every 10s

  onConnect: (frame) => {
    resubscribeAll();             // Re-subscribe to all topics
    requestStateSync();           // Request missed events
  },

  onDisconnect: () => {
    showReconnectingBanner();     // "Reconnecting..."
  },

  onStompError: (frame) => {
    if (frame.headers['message'] === 'TOKEN_EXPIRED') {
      refreshTokenThenReconnect();
    }
  }
});
```

8.2 Exponential Backoff Schedule

RECONNECTION ATTEMPT SCHEDULE:

```
Attempt 1:  Wait 5 seconds (immediate retry)
Attempt 2:  Wait 5 seconds (same – quick recovery)
Attempt 3:  Wait 10 seconds
Attempt 4:  Wait 15 seconds
Attempt 5:  Wait 30 seconds
Attempt 6+: Wait 30 seconds (cap at 30s, keep trying)
```

STOMP client handles this automatically with `reconnectDelay`.
@stomp/stompjs v6+ has built-in exponential backoff.

UI STATES DURING RECONNECTION:

0 seconds:	Connection drops
	→ Toast: "Connection lost. Reconnecting..."
	→ Orange banner appears at top
	→ Data becomes "stale" (grayed overlay)
	→ Last update timestamp shown
5 seconds:	First reconnect attempt
	→ "Attempting to reconnect..." (attempt 1)

5 seconds:	Success (most common case)
	→ Banner:  "Reconnected! Syncing..."
	→ Full state sync requested
	→ Data refreshes
	→ Toast dismissed after 3 seconds
30+ seconds:	Persistent failure
	→ "⚠️ Connection issues. Data may be stale."
	→ Show "Refresh Page" button
	→ REST API polling fallback activates
	→ Poll /api/v1/tournaments/{id}/leaderboard every 10 seconds as fallback

8.3 Post-Reconnection State Sync

STATE SYNCHRONIZATION AFTER RECONNECT:

Step 1: Re-subscribe to all topics:

- /topic/tournament.{id}.leaderboard
- /topic/tournament.{id}.matches
- /topic/tournament.{id}.command (if staff)
- /user/{id}/queue/notifications
- /user/{id}/queue/credentials (if active match)

Step 2: Send sync request:

Client → Server:

/app/sync.request

```
{
  "tournamentId": "tid-001",
  "lastKnownSequenceNumber": 38,    ← Last event sequence received
  "lastKnownLeaderboardVersion": "2025-06-15T14:40:00Z"
}
```

Step 3: Server responds:

Server → Client:

/user/{id}/queue/sync-response

```
{
  "syncType": "FULL_STATE",
  "currentLeaderboard": { ... full leaderboard ... },
  "activeMatches": [ ... all match statuses ... ],
  "pendingActions": { ... if staff ... },
  "missedEvents": [
    { eventCode: "MATCH_STARTED", sequenceNumber: 39, ... },
    { eventCode: "LEADERBOARD_UPDATED", sequenceNumber: 40, ... }
  ],
  "currentSequenceNumber": 48
}
```

Step 4: Client processes sync:

- Apply missed events in sequence order
- Current state guaranteed consistent with server
- Remove "stale" overlay from UI
- Show  "Synced" indicator

MISSED EVENT DETECTION:

Every incoming event has a `sequenceNumber`.

Client tracks `lastReceivedSequenceNumber`.

If `incoming sequenceNumber > lastReceived + 1`:

- Gap detected: missed events during disconnect
- Automatically trigger `sync.request`
- No manual intervention needed

SECTION 9 — DATA SYNCHRONIZATION STRATEGY

9.1 Three-Layer Sync Architecture

DATA SYNCHRONIZATION LAYERS:

LAYER 1: INITIAL LOAD (REST API)

When React component mounts (tournament page, CC, etc.):

- TanStack Query fetches current state via REST API
- REST API reads from Caffeine cache (leaderboard) or directly from MySQL (other data)
- Full current state rendered
- Then STOMP connection established for live updates

WHY REST FIRST:

- └─ Guaranteed current state (no dependency on WebSocket)
- └─ Works even if WebSocket not yet connected
- └─ Faster initial render (cached data)
- └─ SSR-compatible if React ever adds SSR

LAYER 2: REAL-TIME UPDATES (STOMP WebSocket)

After initial load, STOMP keeps data current:

- Leaderboard: `LEADERBOARD_UPDATED` → replace entire array
- Matches: `MATCH_STATUS_CHANGED` → update specific match object
- Check-ins: `CHECKIN_UPDATED` → update specific team in grid
- Notifications: `NOTIFICATION_RECEIVED` → prepend to list

DATA MERGE STRATEGY:

- └─ Leaderboard: REPLACE entire array on every update (no partial merge – prevents state inconsistency)
- └─ Match list: FIND by `matchId`, UPDATE specific object
- └─ Check-in grid: FIND by `registrationId`, UPDATE status
- └─ Notification list: PREPEND new notification

LAYER 3: RECOVERY SYNC (After Reconnect)

After reconnect (Section 8.3):

- `/app/sync.request` sent with `lastKnownSequenceNumber`
- Server sends FULL STATE (not just missed events)

- Client REPLACES all state with server state
- Sequence number aligned

WHY FULL STATE (not event replay):

- └─ Simpler: No need to store full event log
- └─ Safer: No risk of replay order issues
- └─ Fast: JSON of current state is small
- └─ Idempotent: Same result regardless of how many events were missed

9.2 Optimistic Updates

OPTIMISTIC UPDATES IN REACT (for snappy UX):

WHEN TO USE:

Only for low-stakes UI actions, NOT for tournament data.

USED FOR:

- └─ Mark notification as read
 - UI marks read immediately
 - PATCH /notifications/{id}/read in background
 - Rollback if API fails (rare)
- └─ Credential acknowledgment
 - "Got It" button shows  immediately
 - POST /credentials/acknowledge in background
 - └─ Rollback: show "Got It" button again

NEVER USED FOR:

- └─ Score submission (must wait for server confirmation)
- └─ Registration approval (server is authoritative)
- └─ Leaderboard data (always server-authoritative)
- └─ Credential data (security: must be server-confirmed)

9.3 State Freshness Indicators

STALENESS TRACKING IN UI:

Every real-time data component shows:

- └─ Last updated timestamp: "Updated 2 seconds ago"
- └─ Connection status indicator (green dot / orange dot)
 - └─ "Stale" overlay when connection lost > 30 seconds

TIMESTAMP FORMAT:

- └─ < 60 seconds ago: "Just now" or "X seconds ago"
- └─ < 5 minutes ago: "X minutes ago"
- └─ Older: "Last updated at HH:MM:SS"

COMMAND CENTER SPECIFIC:

- └─ Each section header shows connection status
- └─ Disconnected sections: Grey header + "OFFLINE" badge

- └ On reconnect: Headers turn green + "SYNCED" badge
- └ Last sync time always visible in CC header

SECTION 10 — FALLBACK BEHAVIOR

10.1 Fallback Strategy Overview

FALLBACK HIERARCHY:

Level 1: WebSocket (STOMP) – Primary

Best: Real-time, push-based, efficient

Used: When connected

↓ Falls back to ↓

Level 2: SockJS Long Polling – Secondary

Good: Near real-time via polling over HTTP

Used: When WebSocket blocked (OBS, corporate firewalls)

Latency: ~2-4 seconds (vs <1s for WebSocket)

↓ Falls back to ↓

Level 3: REST API Polling – Tertiary

Acceptable: REST endpoint polled periodically

Used: When SockJS also fails (30+ seconds no STOMP)

Latency: Up to 10 seconds

↓ Falls back to ↓

Level 4: Manual Refresh – Last Resort

Minimum: "Page is not live – click to refresh"

Used: When all automated mechanisms fail

Latency: Manual

10.2 SockJS Fallback (OBS Support)

SOCKJS TRANSPORT NEGOTIATION:

OBS Studio uses Chromium browser engine.

Standard WebSocket works in OBS.

But corporate/restricted network OBS machines may block WS.

SockJS negotiation sequence:

1. Client attempts WebSocket upgrade

└ Success → Use WebSocket (fastest)

2. WebSocket fails:

Client attempts xhr-streaming

(HTTP streaming – server keeps connection open)

└ Success → Use xhr-streaming

3. xhr-streaming fails:
Client attempts iframe-eventsourcing
(EventSource via hidden iframe)
└─ Success → Use EventSource
4. All streaming fails:
Client falls back to xhr-polling
(Standard HTTP long-polling)
Poll interval: 1 second
Latency impact: 1-2 seconds additional delay
└─ Acceptable for OBS overlay (still fast enough)

REACT CLIENT CONFIGURATION:

```
websocketFactory: () => new SockJS('https://api.gameverse.gg/ws')  
→ SockJS auto-negotiates best transport  
→ Same @stomp/stompjs client code for all transports  
→ Transparent to React components
```

10.3 REST Polling Fallback

REST POLLING FALLBACK ACTIVATION:

Trigger: WebSocket disconnected for > 30 seconds
AND reconnection attempts exhausted

POLLING INTERVALS:

```
└─ Leaderboard:      Every 10 seconds  
  GET /api/v1/tournaments/{id}/leaderboard  
  → TanStack Query: { refetchInterval: 10000 }  
└─ Match statuses:  Every 10 seconds  
  GET /api/v1/tournaments/{id}/matches  
  → TanStack Query: { refetchInterval: 10000 }  
└─ Notifications:   Every 30 seconds  
  GET /api/v1/notifications?unread=true  
  → TanStack Query: { refetchInterval: 30000 }  
└─ Pending actions: Every 10 seconds (Command Center)  
  GET /api/v1/tournaments/{id}/command-center/overview  
  → TanStack Query: { refetchInterval: 10000 }
```

UI INDICATOR:

Orange banner: "⚠️ Live updates unavailable. Refreshing every 10 seconds."
Show last REST poll timestamp.

DEACTIVATION:

When STOMP reconnects successfully:
→ Clear polling intervals (refetchInterval: false)
→ Request full state sync via STOMP
→ Banner: ✅ "Live updates restored"
→ Dismiss banner after 5 seconds

REACT IMPLEMENTATION:

```
const { data, isPolling } = useQuery({
  queryKey: ['leaderboard', tournamentId],
  queryFn: fetchLeaderboard,
  refetchInterval: stompConnected ? false : 10000,
  staleTime: stompConnected ? Infinity : 5000,
});
```

10.4 Offline Mode (Player Dashboard)

OFFLINE BEHAVIOR FOR PLAYERS:

Critical for match day: Player may have spotty mobile data.

What works offline (Service Worker cached):

- └─ Tournament schedule (cached on last load)
- └─ Team roster (cached)
- └─ Match timing (cached)
- └─ Tournament rules (cached)

What requires connectivity:

- └─ Live leaderboard (cannot show without WS or polling)
- └─ Room credentials (NEVER cached – security requirement)
- └─ Real-time match status
- └─ Notifications

OFFLINE CREDENTIAL BEHAVIOR:

If player loses connectivity AFTER credentials were displayed:

- React stores credentials in component state (in-memory)
- As long as app stays open, credentials remain visible
- App does NOT cache credentials to localStorage/sessionStorage
- Credential card shows: "⚠️ Offline – credentials shown may be outdated"
- Expires countdown continues (based on stored expiresAt)

If player loses connectivity BEFORE credentials released:

- CRITICAL PATH: Credential delivery may fail
- SMS fallback for team captain (always sent for credentials)
- Referee's Lobby Readiness Tracker shows captain "Not Acknowledged"
- Referee can manually share credentials via external channel
- Tournament Director alerted

PUSH NOTIFICATION AS BACKUP:

FCM/APNs push notifications are server-initiated.

They deliver even if app is not open.

For credentials: Push notification sent simultaneously with STOMP.

Player opens notification → App loads → Credentials fetched via REST API.

10.5 OBS Overlay Fallback

OBS OVERLAY SPECIFIC FALLBACKS:

OBS Browser Source behavior on connection loss:

Scenario A: STOMP disconnects mid-tournament

- Overlay JavaScript: onDisconnect() triggers
- Connection status dot: ● Red (bottom-right corner)
- Data display: FROZEN at last received state
(Better than blank – broadcast continues normally)
- Reconnection: Automatic via SockJS backoff
- On reconnect: Full state sync requested
- Data: Updates to current state
- Status dot: ● Green

Scenario B: Complete network failure on production PC

- OBS Browser Source loses all network access
- Overlay shows: Last known state (frozen)
- Connection status: ● Red
- Production team fallback: Switch to "Break Screen" scene in OBS
(Pre-configured static tournament graphic – no data dependency)
- On network restore: Browser Source auto-reconnects

Scenario C: Spring Boot server maintenance/restart

- Server sends STOMP server shutdown notice (if graceful)
- All overlays disconnect simultaneously
- 15-second window while server restarts (Spring Boot ~10s startup)
- Overlays show: Frozen last state
- Auto-reconnect restores within 15-30 seconds
- Full sync on reconnect: Overlays update to current state

THE FROZEN STATE PRINCIPLE:

An overlay showing 5-minute-old data during a brief outage is infinitely better than an overlay showing a blank/error screen. Overlays NEVER show error messages or blank states. They always show the last known valid state.

OVERLAY RECONNECTION INDICATOR:

Tiny colored dot in corner of overlay (configurable to hide):

- ● Solid green: Connected, real-time
- ● Yellow: Reconnecting
- ● Red: Disconnected (data frozen)

Broadcast producer can toggle visibility in Broadcast Dashboard.

SECTION 11 — CHANNEL & SUBSCRIPTION ARCHITECTURE

11.1 Channel Permission Matrix

WHO CAN SUBSCRIBE TO WHAT:

STOMP Destination	Allowed Subscribers
-------------------	---------------------

/topic/tournament.{id}.leaderboard	Everyone (public topic)
/topic/tournament.{id}.matches	Everyone (public topic)
/topic/tournament.{id}.announce	Authenticated users
/topic/tournament.{id}.status	Everyone (public topic)
/topic/tournament.{id}.stream	Broadcast Producer + Director
/topic/tournament.{id}.overlays	Overlay tokens + Staff
/topic/tournament.{id}.checkins	Tournament staff only
/topic/tournament.{id}.command	Tournament staff only
/topic/tournament.{id}.disputes	Tournament Director + Org Owner

/user/{id}/queue/notifications	Own user_id only
/user/{id}/queue/credentials	Own user_id only
/user/{id}/queue/team	Own user_id only
/user/{id}/queue/sync-response	Own user_id only

ENFORCEMENT:

Spring ChannelInterceptor.preSend():

- └ On SUBSCRIBE frame: check destination vs user's permissions
- └ Use same RBAC logic as REST API (@Service layer)
- └ Denied subscriptions: STOMP ERROR frame returned
- └ Logging: All subscription attempts logged for audit

STAFF TOPIC VALIDATION:

/topic/tournament.{id}.command subscription attempt:

- Extract userId from WebSocket session
- Query DB: is user TOURNAMENT_DIRECTOR, REFEREE, or ORG_OWNER for this tournament_id?
- YES: Allow subscription
- NO: STOMP ERROR { message: "ACCESS_DENIED" }

11.2 Message Flow Per Consumer Type

MESSAGE ROUTING PER CONSUMER:

VIEWER (watching public tournament page):

Subscriptions:

- └ /topic/tournament.{id}.leaderboard ← Leaderboard tab
- └ /topic/tournament.{id}.matches ← Schedule tab
- └ /topic/tournament.{id}.status ← Tournament header

Receives:

- └ LEADERBOARD_UPDATED 
- └ MATCH_STATUS_CHANGED 
- └ RESULT_PUBLISHED  (match results visible)
- └ MATCH_PAUSED  (public message shown)
- └ TOURNAMENT_STATUS_CHANGED 

Does NOT receive:

- └ DISPUTE_CREATED 
- └ CREDENTIALS_RELEASED 
- └ CHECKIN_UPDATED 
- └ PENDING_ACTIONS_UPDATED 

PLAYER / TEAM CAPTAIN (match day dashboard):

Subscriptions (all viewer channels PLUS):

- └ /user/{id}/queue/notifications ← Personal notifications
- └ /user/{id}/queue/credentials ← Match credentials
- └ /user/{id}/queue/team ← Team updates

Receives additionally:

- └ CREDENTIALS_RELEASED ✓ (own match only)
- └ NOTIFICATION_RECEIVED ✓ (personal)
- └ REGISTRATION_STATUS_CHANGED ✓ (own registration)
- └ PRIZE_PAYOUT_READY ✓ (if winner)

TOURNAMENT DIRECTOR (Command Center):

Subscriptions (all player channels PLUS):

- └ /topic/tournament.{id}.command ← Staff events
- └ /topic/tournament.{id}.checkins ← Check-in grid
- └ /topic/tournament.{id}.stream ← Stream health

Receives additionally:

- └ PENDING_ACTIONS_UPDATED ✓
- └ DISPUTE_CREATED ✓
- └ RESULT_SUBMITTED ✓ (pending verification)
- └ DQ_RECOMMENDED ✓
- └ CHECKIN_UPDATED ✓
- └ STREAM_HEALTH_UPDATE ✓

BROADCAST PRODUCER (Broadcast Dashboard):

Subscriptions:

- └ /topic/tournament.{id}.leaderboard ← Preview
- └ /topic/tournament.{id}.matches ← Match status
- └ /topic/tournament.{id}.stream ← Stream health
- └ /topic/tournament.{id}.overlays ← Overlay commands

Receives:

- └ LEADERBOARD_UPDATED ✓ (preview pane)
- └ STREAM_HEALTH_UPDATE ✓
- └ OVERLAY_COMMAND ✓ (echoed back for confirm)
- └ MATCH_STATUS_CHANGED ✓

OBS BROWSER SOURCE (Overlay):

Connection: Authenticated with overlay token (not user JWT)

Subscriptions:

- └ /topic/tournament.{id}.leaderboard
- └ /topic/tournament.{id}.matches
- └ /topic/tournament.{id}.overlays

Receives:

- └ LEADERBOARD_UPDATED ✓ → Updates overlay display
- └ RESULT_PUBLISHED ✓ → Triggers result splash
- └ MATCH_STATUS_CHANGED ✓ → Updates match bar
- └ OVERLAY_COMMAND ✓ → Show/hide/scene commands

SECTION 12 — OBS OVERLAY REAL-TIME SYSTEM

12.1 Overlay Architecture Deep Dive

OBS OVERLAY REAL-TIME SYSTEM:

OVERLAY PAGE LIFECYCLE:

1. OBS adds Browser Source URL:
`https://gameverse.gg/overlay/{tournamentId}/leaderboard?token={token}`
2. Nginx serves static HTML (5-min CDN cache):
 - └─ Minimal HTML shell
 - └─ Embedded CSS (no external requests)
 - └─ Embedded initial tournament state (injected by Spring Boot at token validation time – rendered into HTML template)
 - └─ JavaScript: STOMP client + overlay renderer
3. JavaScript initializes:
 - └─ Parse `window.__INITIAL_STATE__` (initial leaderboard data)
 - └─ Render initial state to DOM immediately (no flash)
 - └─ Connect STOMP via SockJS
 - └─ Subscribe to relevant topics
 - └─ Start heartbeat (overlay stays alive during long sessions)
4. On `LEADERBOARD_UPDATED` received:
 - └─ Parse `event.data.leaderboard` array
 - └─ For each team:
 - └─ Find or create DOM row
 - └─ Update: rank number, team name, points, kills
 - └─ If `rankChange > 0`: add CSS class 'rank-up' (↑ green)
 - └─ If `rankChange < 0`: add CSS class 'rank-down' (↓ red)
 - └─ CSS transition: `translateY` animates row to new position
 - └─ Rows animate to sorted positions (600ms transition)
5. On `OVERLAY_COMMAND: { command: SHOW, overlayType: RESULT_SPLASH }`:
 - └─ Render result splash with top 3 teams
 - └─ CSS animation: slides in from bottom
 - └─ Display for `params.displayDurationSeconds` (15s)
 - └─ CSS animation: slides out → auto-hides
6. On `OVERLAY_COMMAND: { command: HIDE }`:
 - └─ CSS: `opacity 0`, `display none`

RENDERING APPROACH (Vanilla JS, no React):

OBS Browser Sources benefit from lightweight rendering.
No React virtual DOM overhead in overlay pages.
Direct DOM manipulation for smooth 60fps animations.
CSS transitions handle all animation (GPU-accelerated).

12.2 Overlay Token Security

OVERLAY TOKEN DESIGN:

Purpose: Allow OBS to connect without a user JWT
(OBS has no user account)

Token Generation:

- └─ Generated when Broadcast Producer opens Overlay Management
- └─ UUID v4 (256-bit entropy)
- └─ Stored in overlay_configs table with:
 - └─ tournament_id
 - └─ overlay_type
 - └─ token_hash (SHA-256 of token)
 - └─ created_at
 - └─ expires_at (tournament end + 30 days)
- └─ Token included in overlay URL as query param

Token Validation (on STOMP CONNECT):

- └─ Extract token from CONNECT headers
- └─ SHA-256 hash the token
- └─ Lookup in overlay_configs by hash
- └─ Verify: not expired, tournament matches
- └─ Create OverlayAuthentication (not UserAuthentication)

OverlayAuthentication grants:

- └─ Subscribe to: /topic/tournament.{id}.leaderboard ✓
- └─ Subscribe to: /topic/tournament.{id}.matches ✓
- └─ Subscribe to: /topic/tournament.{id}.overlays ✓
- └─ Cannot subscribe to: any user queues, command topics ✗

Token Rotation:

If Broadcast Producer rotates overlay token:

- └─ Old token: is_active = false
- └─ New token: generated and returned
- └─ OBS: Producer must update URL in OBS
- └─ Old overlays: Disconnected immediately on next heartbeat check

SECTION 13 — SECURITY IN REAL-TIME

13.1 Security Threat Model for WebSocket

THREAT MODEL & MITIGATIONS:

THREAT 1: Credential Eavesdropping

Risk: Attacker intercepts WebSocket traffic to steal room credentials.

Mitigation:

- └─ TLS 1.3 end-to-end (wss://) – traffic encrypted in transit
- └─ Credentials sent to /user/{id}/queue/ – user-specific delivery
- └─ Spring STOMP: User destinations isolated per session
- └─ Credentials expire after 30 minutes

Risk Level: LOW (mitigated by TLS + user isolation)

THREAT 2: Unauthorized Subscription to Private Topics

Risk: Attacker subscribes to /topic/tournament.{id}.command

to see dispute details and staff communications.

Mitigation:

- └ ChannelInterceptor validates every SUBSCRIBE frame
- └ DB-checked RBAC (same as REST API)
- └ ERROR frame returned immediately for denied subscriptions
- └ Attempt logged to audit_log

Risk Level: LOW (enforced at broker level)

THREAT 3: Fake Score Injection via WebSocket

Risk: Attacker sends a STOMP message to fake a leaderboard update.

Mitigation:

- └ Leaderboard is ONLY updated by server-side Spring service
- └ No client-to-server STOMP message affects leaderboard
- └ @MessageMapping on /app/* validates auth + RBAC strictly
- └ Client CANNOT publish to /topic/ directly
- └ Only Spring SimpMessagingTemplate can publish to /topic/

Risk Level: NONE (architecture prevents this by design)

THREAT 4: WebSocket Session Hijacking

Risk: Attacker steals WebSocket session to receive another user's creds.

Mitigation:

- └ JWT required on CONNECT (not just cookies)
- └ JWT is stateless – no server-side session to steal
- └ /user/{id}/queue/ – Spring maps to principal username from JWT
- └ Even with session ID, cannot access another user's queue
- └ TLS prevents session ID interception

Risk Level: LOW (JWT + TLS mitigated)

THREAT 5: STOMP Message Flooding

Risk: Authenticated user floods /app/chat.send or /app/sync.request.

Mitigation:

- └ ChannelInterceptor rate limit check (per user, per minute)
- └ Chat: 30 messages/minute
- └ sync.request: 5/minute (throttled – full sync is expensive)
- └ Violation: STOMP ERROR + temporary subscription block

Risk Level: LOW (rate limiting mitigated)

THREAT 6: Overlay Token Theft

Risk: Token in OBS URL exposed (OBS properties visible on stream).

Mitigation:

- └ Overlay tokens only grant read access to public-ish data
- └ Cannot subscribe to command/dispute topics with overlay token
- └ Cannot send messages with overlay token
- └ Token can be rotated instantly (old token deactivated)
- └ Overlay data (leaderboard) is already public on website

Risk Level: LOW (read-only, rotatable)

13.2 Credential Isolation Architecture

Room credentials represent competitive integrity.
A leaked credential can allow unauthorized players in the room,
disqualify legitimate teams, or enable match manipulation.

Layer 1: Storage

Layer 2: Delivery Channel

```
/user/{specific-player-id}/queue/credentials
```

Spring STOMP maps `/user/{principal}` to the specific WebSocket session of that authenticated user. No other session receives it.

For each recipient, a SEPARATE payload is constructed:

```
{ roomId: "A7B3C9", password: "XK29", slotNumber: 3 }
```

This ensures delivery is personalized and targeted.

Decryption happens inside `CredentialReleaseService.release()`.

No plaintext written to logs, databases, or cache.

Every credential view is logged:

Referee can see who has and hasn't viewed credentials.

Credentials expire after 30 minutes.

→ Credential Card shows: "❌ Credentials expired"

→ REST API returns 410 Gone for credential requests

→ WebSocket: CREDENTIALS_EXPIRED event sent

- Referee can re-enter new credentials if needed

14.1 Performance Targets & Measurements

PERFORMANCE TARGETS:

WEBSOCKET CONNECTIONS:

- └ Phase 1 Target: 500 concurrent connections per instance
- └ Phase 2 Target: 5,000 concurrent connections (multi-instance)
- └ Java 21 Virtual Threads: Each connection = lightweight virtual thread
- └ Tomcat + Spring WebSocket: Well-tested to 1,000+ concurrent WS

MESSAGE THROUGHPUT:

- └ Leaderboard updates: 1 broadcast per match result (every ~30min)
 - └ → 64 recipients per tournament × N tournaments
 - └ → Total: ~320 messages per leaderboard update (5 tournaments)
- └ Credential delivery: 64 messages per match × 16 matches = 1,024/day
- └ Notifications: ~2,000/tournament distributed over 8 hours
- └ Command Center: ~100 events/tournament to ~5 staff recipients

LATENCY BUDGET (Score → OBS, 3s total):

- └ REST API validation: ~50ms
 - └ DB transaction: ~30ms
 - └ Domain event dispatch: ~5ms
 - └ Leaderboard calculation: ~50ms
 - └ DB leaderboard update: ~30ms
 - └ Cache invalidation: ~5ms
 - └ STOMP broadcast: ~10ms
 - └ Network delivery: ~50ms (India RTT)
 - └ Browser render + animate: ~100ms
- Total: ~330ms actual | Budget: 3,000ms | Headroom: 9x ✓

PAYLOAD SIZES (important for mobile):

- └ LEADERBOARD_UPDATED (64 teams, no breakdown): ~8KB
- └ LEADERBOARD_UPDATED (64 teams, with breakdown): ~25KB
- └ MATCH_STATUS_CHANGED: ~500B
- └ CREDENTIALS_RELEASED: ~600B
- └ PENDING_ACTIONS_UPDATED: ~2KB
- └ NOTIFICATION_RECEIVED: ~400B

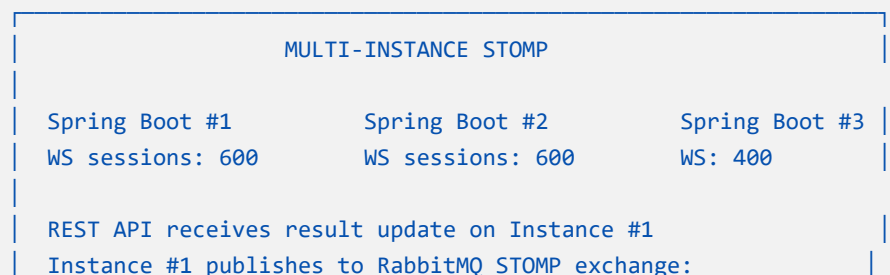
14.2 Phase 2 Scaling Architecture

SCALING BEYOND SINGLE INSTANCE:

TRIGGER: Multiple Spring Boot instances needed

(When: > 10 concurrent live tournaments, > 2,000 WebSocket connections)

CHANGE: Add RabbitMQ with STOMP Plugin



```
"leaderboard.updated" message
```

```
RabbitMQ STOMP relay:
```

- Instance #1 subscribers receive message
- Instance #2 subscribers receive message
- Instance #3 subscribers receive message

```
ALL 1,600 connected clients receive update simultaneously
```

SPRING CONFIGURATION CHANGE (no code change):

application.yml:

spring:

websocket:

relay:

```
host: rabbitmq.internal
port: 61613
login: gameverse
passcode: ${RABBITMQ_PASS}
virtualHost: /gameverse
```

(Replaces in-memory simple broker with RabbitMQ relay.
Same @SendTo annotations, same SimpMessagingTemplate.
Zero application code changes.)

14.3 Leaderboard Calculation Optimization

LEADERBOARD CALCULATION PERFORMANCE:

PHASE 1 (MySQL query, 64 teams, 16 matches):

Average query time: ~20-40ms (well within budget)

Query optimization:

- └ Composite index: (tournament_id, submission_status) on match_results
- └ Composite index: (result_id) on team_match_results
- └ HikariCP: Dedicated connection pool for leaderboard executor
- └ Read replica: Leaderboard query runs on read replica (Phase 2)

PHASE 2 (Pre-computed, Redis-cached):

After leaderboard recalculation:

- Store full sorted leaderboard JSON in Redis (60s TTL)
- REST API GET /leaderboard reads from Redis (1-2ms)
- STOMP broadcast contains full payload from Redis
- No DB query on broadcast path

INCREMENTAL UPDATES (Phase 3 optimization):

Instead of broadcasting full 64-team array:

- Only send teams that CHANGED rank or points
- Client merges partial update into existing state
- Reduces payload: 64 teams → typically 3-5 changed teams
- Bandwidth reduction: 8KB → ~1KB per update
- Implement when: > 50 tournaments with > 500 viewers each

SECTION 15 — ERROR HANDLING IN REAL-TIME

15.1 Server-Side Error Handling

SERVER-SIDE REAL-TIME ERROR SCENARIOS:

SCENARIO 1: Leaderboard calculation fails

Cause: DB timeout, constraint violation, OOM

Action:

- └─ Exception caught in @Async leaderboard executor
- └─ Retry: 3 attempts with 1s backoff
- └─ If all fail: Alert via Sentry (P1)
- └─ No LEADERBOARD_UPDATED sent (clients keep previous state)
- └─ Director receives: STAFF_ALERT { type: LEADERBOARD_CALC_FAILED }
- └─ REST API: GET /leaderboard returns last cached state

SCENARIO 2: Credential delivery partially fails

Cause: Some WebSocket sessions inactive/closed

Action:

- └─ SimpMessagingTemplate delivery: best-effort per session
- └─ For each failed delivery: log to credential_logs (NOT_DELIVERED)
- └─ Fallback: Push notification sent simultaneously (FCM/APNs)
- └─ Fallback: SMS to team captain (always sent for credentials)
- └─ Referee alert: STAFF_ALERT { teamsNotReceived: [list] }
- └─ Manual intervention: Referee can resend or rotate

SCENARIO 3: STOMP message too large

Cause: Leaderboard with breakdown for 128 teams might exceed limits

Action:

- └─ STOMP frame limit: 64KB (configurable)
- └─ Mitigation: Strip match_breakdown from real-time payload
- └─ (breakdown available via REST API on demand)
- └─ Message size budget enforced per event type
- └─ If exceeded: Log warning + send stripped payload

SCENARIO 4: Duplicate event delivery

Cause: Race condition – two events trigger same broadcast

Action:

- └─ Idempotency: eventId UUID in every event envelope
- └─ React client: deduplicate by eventId (last 50 event IDs in Set)
- └─ OBS overlay: same deduplication in JavaScript
- └─ Effect: Client ignores duplicate, no visual glitch

15.2 Client-Side Error Handling (React)

CLIENT-SIDE REAL-TIME ERROR HANDLING:

STOMP ERROR Frame received:

```
switch (error.headers.message) {
  case 'TOKEN_EXPIRED':
    refreshTokenThenReconnect();
    break;

  case 'ACCESS_DENIED':
    showToast("You don't have permission to view this.");
    unsubscribeFromTopic(destination);
    break;

  case 'TOURNAMENT_NOT_FOUND':
    showToast("Tournament no longer available.");
    navigate('/explore');
    break;

  case 'RATE_LIMIT_EXCEEDED':
    showToast("Sending messages too fast. Slowing down.");
    enableThrottling();
    break;

  default:
    Sentry.captureMessage('Unknown STOMP error', error);
    showToast("Connection error. Reconnecting...");
}
```

MALFORMED MESSAGE received:

```
try {
  const event = JSON.parse(message.body);
  processEvent(event);
} catch (e) {
  Sentry.captureException(e, { extra: { rawBody: message.body } });
  // Silently ignore – don't crash the component
  // REST API polling will catch any missed state
}
```

SEQUENCE NUMBER GAP detected:

```
if (event.sequenceNumber > lastSequenceNumber + 1) {
  // Gap detected: missed events during brief disconnect
  requestStateSync(lastSequenceNumber);
  // Continue processing current event
  // Sync response will fill the gaps
}
lastSequenceNumber = event.sequenceNumber;
```

SECTION 16 — TESTING REAL-TIME SYSTEMS

16.1 Testing Strategy

REAL-TIME TESTING PYRAMID:

UNIT TESTS (JUnit 5 + Mockito):

- └─ ScoringEngine.calculate() – all placement + kill combinations
- └─ LeaderboardEngine.recalculate() – rank ordering, tiebreakers
- └─ CredentialEncryption – encrypt/decrypt round trip
- └─ EventPayloadBuilder – correct payload construction per event type
- └─ ChannelInterceptor – permission checks per destination

INTEGRATION TESTS (Spring Boot Test + Testcontainers):

- └─ Full result submission → DB → event → leaderboard recalc
- └─ Credential encryption → store → decrypt → deliver
- └─ Tournament status state machine transitions
- └─ Payment webhook → registration confirmation
- └─ Dispute creation → director notification event

WEBSOCKET INTEGRATION TESTS (Spring Boot Test + STOMP test client):

- └─ Connect with valid JWT → CONNECTED frame received
- └─ Connect with expired JWT → ERROR frame returned
- └─ Subscribe to public topic → subscription accepted
- └─ Subscribe to staff topic as player → subscription denied
- └─ Submit result → LEADERBOARD_UPDATED received on topic
- └─ Release credentials → CREDENTIALS_RELEASED received by player
- └─ Release credentials → NOT received by other players
- └─ Reconnect after disconnect → state sync successful
- └─ Sequence number gap → sync request triggered

LOAD TESTS (JMeter / Gatling):

- └─ 500 concurrent WebSocket connections
- └─ 10 simultaneous leaderboard broadcasts
- └─ Credential delivery to 64 simultaneous users
- └─ Measure: Message delivery latency at p50, p95, p99
- └─ Target: p99 < 1000ms for WebSocket delivery

END-TO-END TESTS (Playwright):

- └─ Full tournament day simulation:
 - └─ Open browser (simulating viewer)
 - └─ Navigate to public tournament page
 - └─ Submit result via Referee session
 - └─ Assert: Leaderboard on viewer page updates within 3 seconds
 - └─ Assert: Rank change animations play correctly
 - └─ Assert: OBS overlay page updates simultaneously
- └─ Credential delivery E2E:
 - └─ Referee releases credentials
 - └─ Player session: Assert credential card appears < 2 seconds
 - └─ Player taps acknowledge
 - └─ Referee: Assert lobby readiness shows player acknowledged
- └─ Reconnection E2E:
 - └─ Establish connection
 - └─ Kill WebSocket connection (simulate network drop)
 - └─ Assert: "Reconnecting..." banner appears

- └─ Wait for reconnect (5 seconds)
- └─ Submit result during disconnect
- └─ Assert: After reconnect, leaderboard shows updated state

16.2 Tournament Day Simulation Test

FULL TOURNAMENT DAY SIMULATION (Integration Test):

Test: BGMI 4-match tournament with 16 teams

Setup:

- └─ Create tournament, 16 teams registered + approved
- └─ 4 matches scheduled (4 teams each for test simplicity)
- └─ Director, 2 Referees, Broadcast Producer connected
- └─ 100 simulated viewer WebSocket connections
- └─ 2 OBS overlay connections

Test Steps:

Step 1: Check-in (10 teams check in, 6 don't)

- Assert: CHECKIN_UPDATED sent to staff 10 times
- Assert: 6 teams marked no-show
- Assert: Waitlist teams promoted (if configured)

Step 2: Match 1 lobby opened

- Assert: MATCH_STATUS_CHANGED(LOBBY_OPEN) sent to all 100 viewers
- Referee enters credentials → CREDENTIALS_RELEASED to 16 players
- Assert: 16 CREDENTIALS_RELEASED events sent (not 1, not 100)
- Assert: 0 events sent to 84 other players

Step 3: Match 1 result submitted

- Assert: RESULT_SUBMITTED sent to staff only (100 viewers: 0)
- Director verifies
- Assert: RESULT_PUBLISHED sent to all 100 viewers
- Assert: LEADERBOARD_UPDATED sent within 500ms of publish
- Assert: All 100 viewer connections receive leaderboard update
- Assert: Both OBS overlay connections receive update
- Assert: Rank changes correctly calculated

Step 4: Repeat for matches 2-4

Step 5: Tournament completed

- Assert: TOURNAMENT_STATUS_CHANGED(COMPLETED) sent to all
- Assert: LEADERBOARD_LOCKED sent (isLocked: true)
- Assert: PRIZE_PAYOUT_READY sent to winner users only

Step 6: Reconnection test

- Kill 50 viewer connections mid-tournament
- Simulate Match 3 result during disconnect
- Reconnect 50 viewers
- Assert: All 50 receive sync response with Match 3 result
- Assert: Leaderboard shows correct post-Match-3 standings

PASS CRITERIA:

- └ All 100 viewers receive LEADERBOARD_UPDATED within 3s of result submit
- └ Credential delivery: 100% to 16 players, 0% to others
- └ State after reconnect: Matches server state exactly
- └ No events delivered to unauthorized subscribers
- └ Sequence numbers: No gaps in final event sequence

DOCUMENT COMPLETION SUMMARY

Coverage Checklist

Requirement	Covered	Section
Score Updated → Backend Validation	✓	Section 6.1, Stage 3
Backend Validation → Database Transaction	✓	Section 6.1, Stage 4
Database Transaction → Leaderboard Calculation	✓	Section 6.1, Stage 6
Leaderboard Calculation → Publish Event	✓	Section 6.1, Stage 7
Publish Event → WebSocket	✓	Section 6.1, Stage 7
WebSocket → Public Website	✓	Section 6.1, Stage 8
WebSocket → OBS Overlay	✓	Sections 6.1, 11, 12
WebSocket Events (complete catalog)	✓	Section 4
Event Payloads (all events)	✓	Section 5
Reconnection Behavior	✓	Section 8
Data Synchronization	✓	Section 9
Fallback Behavior	✓	Section 10
Credential Security	✓	Sections 6.2, 13
OBS Overlay Real-Time	✓	Section 12
Performance & Scalability	✓	Section 14
Error Handling	✓	Section 15
Testing Strategy	✓	Section 16

Document Statistics

Element	Count
Event Types Defined	35+

Element	Count
Complete Payload Schemas	14
STOMP Channels	15+
Fallback Levels	4
Pipeline Stages	8
Security Threats Addressed	6
Test Categories	5

Document Sign-Off

Role	Name	Status
Engineering Lead	—	Pending Review
Backend Lead (Spring Boot)	—	Pending Review
Frontend Lead (React)	—	Pending Review
Security Review	—	Pending Review
DevOps Lead	—	Pending Review

Document:

GameVerse Real-Time Architecture

|

Version:

1.0

|

Stack:

React 19 + Java 21 Spring Boot + STOMP

|

Status:

Complete

✔